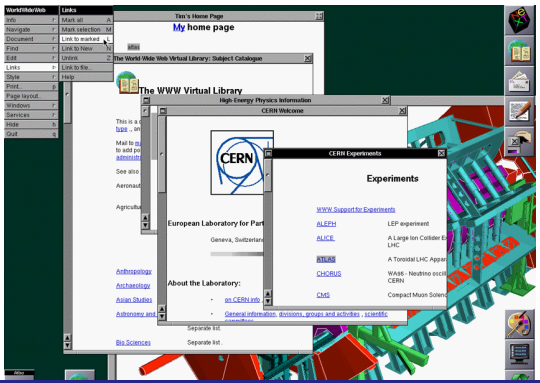
Redes de Datos II
Servicios TCP/IP - Hypertext Transfer Protocol (HTTP/1.1)

Matías Robles

LINTI–UNLP

22 de octubre de 2026



Contenidos

1. Introducción
2. Mensajes y Métodos
3. Funcionamiento
4. HTTP Seguro
5. Cookies
6. HTTP Caching

Estamos en:

1 Introducción

2 Mensajes y Métodos

3 Funcionamiento

4 HTTP Seguro

5 Cookies

6 HTTP Caching

Historia I

1989: Aparece la primera propuesta *moderna* sobre el Web en el CERN (Centro Europeo de Investigación Nuclear) realizada por el físico Tim Berners-Lee.

1993: Los directores del CERN declaran que la tecnología WWW será de libre uso y que no será necesario el pago de licencias.

1993: Apareció el interfaz gráfico Mosaic y su autor Marc Andreessen abandonó el NCSA (National Center for Supercomputers Applications) para fundar Netscape Communications Corp.

1994: El CERN y el MIT (Massachussets Institute of Technology) crean el W3C (WWW Consortium) para estandarizar servicios e incrementar la interoperabilidad.

Historia II

1994: El MIT lleva la parte de EEUU y el centro francés INRIA la parte europea. http://www.w3.org.

1995: Aparece el navegador Netscape.

1996-1997: HTTP 1.1, HTML 3.0

1998: AOL (America On-Line) y Sun compran Netscape Comm. Corp

1999: XML, RDF.

2000: Servicios Web (SOAP, WSDL, UDDI, etc.).

2015: HTTP/2. RFC 7540. Hecha obsoleta por RFC 9113.

2022: HTTP/3. RFC 9114

Introducción

- Actualmente, millones de recursos se mueven en Internet: página web, imágenes JPEG, páginas HTML, archivos XML, archivos de audio, etc.
- HTTP es el protocolo encargado de transferir esos recursos.
- Contenido se almacena en servidores web que hablan el protocolo HTTP, servidores HTTP.
- Clientes realizan solicitudes HTTP para obtener esos recursos.
- Web browsers son los clientes más comunes, aunque no los únicos.
- Además de objetos estáticos, recurso también es el contenido generado dinámicamente por un servidor en base a información que reciben.

Recursos

- Un objetivo en una solicitud HTTP se lo conoce como **recurso**.
- Cada uno de esos recursos tiene un nombre.
- Recursos son identificados por una URI (Uniform Resource Identifier).
- URIs tienen dos formato: URL (Uniform Resource Locator) y URN (Uniform Resource Name)
- URL describe la ubicación específica de un recurso.
- http://www.unlp.edu.ar/home/index.html
  - **http://** se está usando el protocolo http.
  - **www.unlp.edu.ar** servidor al que se le realiza el pedido.
  - **/home/index.html** recurso que el cliente está solicitando.
- URN define un nombre único para un recurso, independietemente de donde se encuentre.
- urn:ieft:rfc:9241

Funcionamiento

- HTTP es un protocolo de solicitud/respuesta sin estado.
- Es un protocolo de texto plano. Usa MIME para enviar datos binarios.
- Modelo cliente/servidor que requiere un transporte confiable.
- Corre sobre TCP y, por defecto, uso el puerto 80.
- Existe una versión segura, HTTPS, usando el protocolo Transport Layer Secuirty (TLS). Por defecto, usa el puerto 443.
- Cliente solicita un recurso y espera la respuesta del servidor. No contienen un identificador que permita relacionarlas.
- HTTP/1.1 permite el *pipelining*, aunque no ha sido muy utilizado.

HTTP - Versiones anteriores

- HTTP/0.9:
  - Primera especificación de HTTP, 1991. Menos de 700 palabras.
  - Especifica que la conexión debe ser sobre TCP (o similar) y al puerto 80.
  - Define un único método, GET, para hacer el requerimiento: *GET /home/index.html*.
  - Servidor responde y luego cierra la conexión.
- HTTP/1.0:
  - Primera RFC en 1995, pero como un documento informativo.
  - Agrega nuevos métodos: POST, HEAD y describe otros: PUT, DELETE, LINK y UNLINK.
  - Incorpora cabeceras HTTP para proveer información adicional al intercambio.
  - Código de 3 dígitos indicando el resultado de la respuesta.
  - Admite MIME y nuevo juego de caracteres (además de ASCII): ISO-8851-1, UTF-8, etc.

Estamos en:

1 Introducción

2 Mensajes y Métodos

3 Funcionamiento

4 HTTP Seguro

5 Cookies

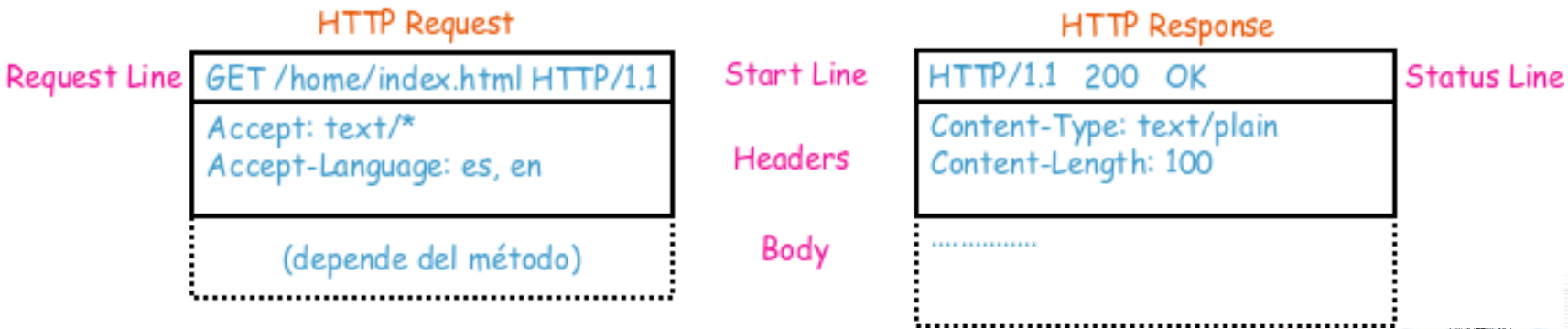
6 HTTP Caching

Mensajes y Métodos

- Clientes envían mensajes de solicitud (request method) al servidor usando un método HTTP.
- Método indica al servidor que acción ejecutar.
- Servidores responden con uno o más mensajes de respuestas (response method) que incluyen un código de estado.
- Con el código de estado se le indica al cliente que sucedió con su solicitud: si fue exitosa o no, si debe realizar alguna acción, etc.
- Además del código, le envía un texto que describe la respuesta.
- En ambos mensajes se pueden incluir cabeceras (headers).

Mensajes y Métodos

- Mensajes tiene misma estructura: línea de inicio, cabeceras y cuerpo (opcional).
- Línea de inicio (start line) es la primera línea de todos los mensajes HTTP.
- Cabeceras(headers) consisten de un nombre y un valor, separados por ":".
- Cabeceras finalizan con una línea en blanco.
- Datos (body) que dependen del método usado y del resultado de la operación.
- Datos puede contener texto o datos binarios.



Métodos

- Indican la operación que se debe ejecutar en el servidor.
- Nombre de los métodos es *case-sensitive*. Letras mayúsculas en US-ASCII.
- *Safe Method:* son de solo lectura. Cliente no espera cambios en el servidor: GET, HEAD, OPTIONS y TRACE.
- *Idempotent Method:* múltiples ejecuciones de una solicitud es igual a ejecutarla una sola vez: PUT, DELETE y safe method.
- En HTTP/1.1, solo es obligatorio implementar los método GET y HEAD.
- Un servidor responde con el código 501(Not Implemented) cuando no reconoce o no tiene implementado el método solicitado.

Métodos (no todos)

- **GET:**
  - Usado por el cliente para solicitarle un recurso al servidor.
  - Permite enviar información, *query string*, como parte de la URL (limitado por las implementaciones).
  - Parámetros van después de **?** y en la forma:

    var1=value,var2=value2.
    - http://www.example.com/index.php?país=Argentina,deporte=tenis
  - Podrían enviarse datos en el body, pero no es lo usual.
- **HEAD:**
  - Idéntico al GET, pero solo se obtienen los metadatos del recurso (headers).
  - No se envía el recurso solicitado en la respuesta.
  - Si la respuesta es dinámica, algunas cabeceras pueden omitirse.

Métodos (no todos) . . .

- **POST:**
  - Permite enviar datos de entrada al servidor, por ej. los campos ingresados en un formulario HTML.
  - Datos se envían en el body del mensajes.
  - Al igual que el GET solicita una respuesta del servidor.
- **PUT:**
  - Permite enviar contenido que será almacenado en el servidor web.
  - Nombre del recurso en el request line indica donde se almacenará el contenido (body) del mensaje.
- **DELETE:**
  - Indica el contenido a ser eliminado en el servidor web.

Estamos en:

1 Introducción

2 Mensajes y Métodos

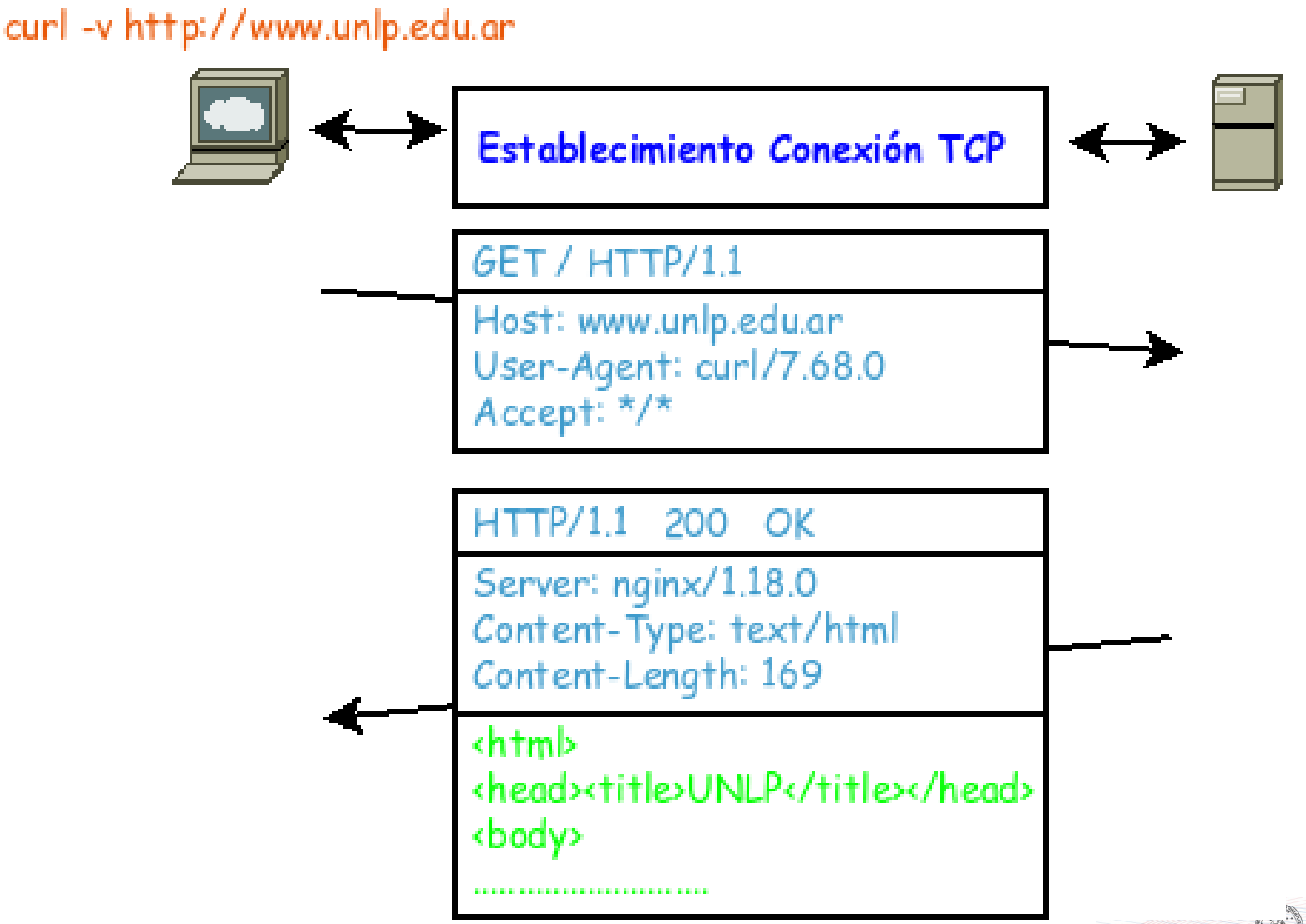
3 Funcionamiento

4 HTTP Seguro

5 Cookies

6 HTTP Caching

¿Cómo funciona http?



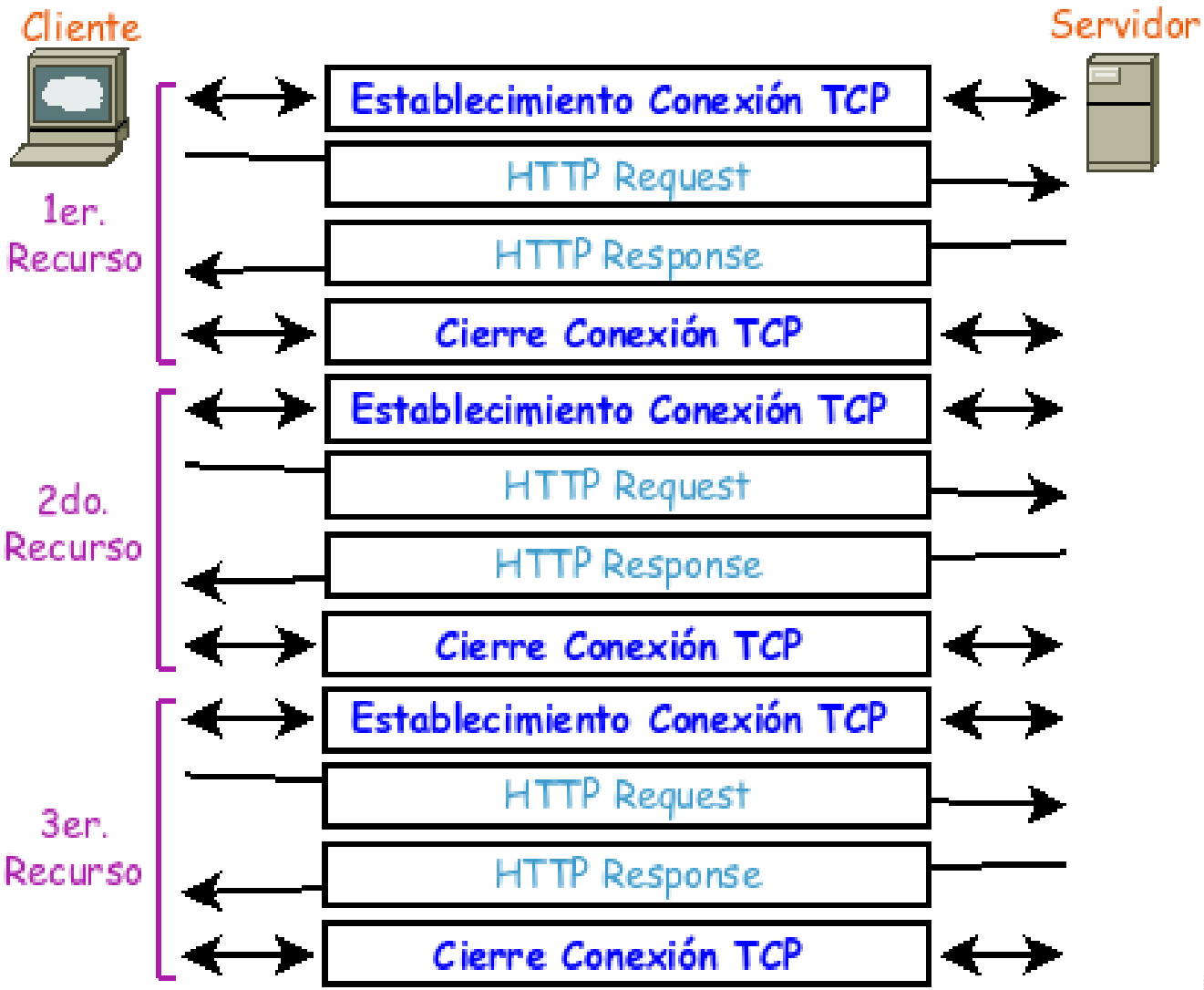
Cabeceras (Headers)

- Cada mensaje puede contener 0, 1 o varias cabeceras.
- Agregan información adicional a los mensaje de solicitud y respuesta.
- Especificación HTTP define varias cabeceras, pero las aplicaciones pueden agregar sus propias cabeceras.
- Son case-insensitive.
- Algunas cabeceras:
  - **Host:** obligatoria en HTTP/1.1. Indica el servidor y el puerto donde se dirige la solicitud.
  - **Date:** provee el día y la hora en que fue creado el mensaje.
  - **Content-Length:** indica la longitud del cuerpo del mensaje.
  - **Content-Type:** tipo de objeto que se está enviando.

Conexiones no persistentes, HTTP/1.0

- Transfiere sólo un objeto por cada sesión.
- Por cada recurso se debe abrir y cerrar una sesión TCP.
- Servidor responde con el objeto y cierra la sesión (cierre sesión TCP)
- Los *browsers* acuden a establecer sesiones en paralelo para reducir el retardo en obtener una página completa
- Algunas implementaciones de HTTP/1.0 fueron extendidas para soportar conexiones persistentes, *keep-alive connections*.
- Cliente puede solicitar que la conexión se mantenga abierta. *Connection: keep-alive*.
- Servidor responde con esa misma cabecera si lo acepta. Caso contrario, cierra la conexión.

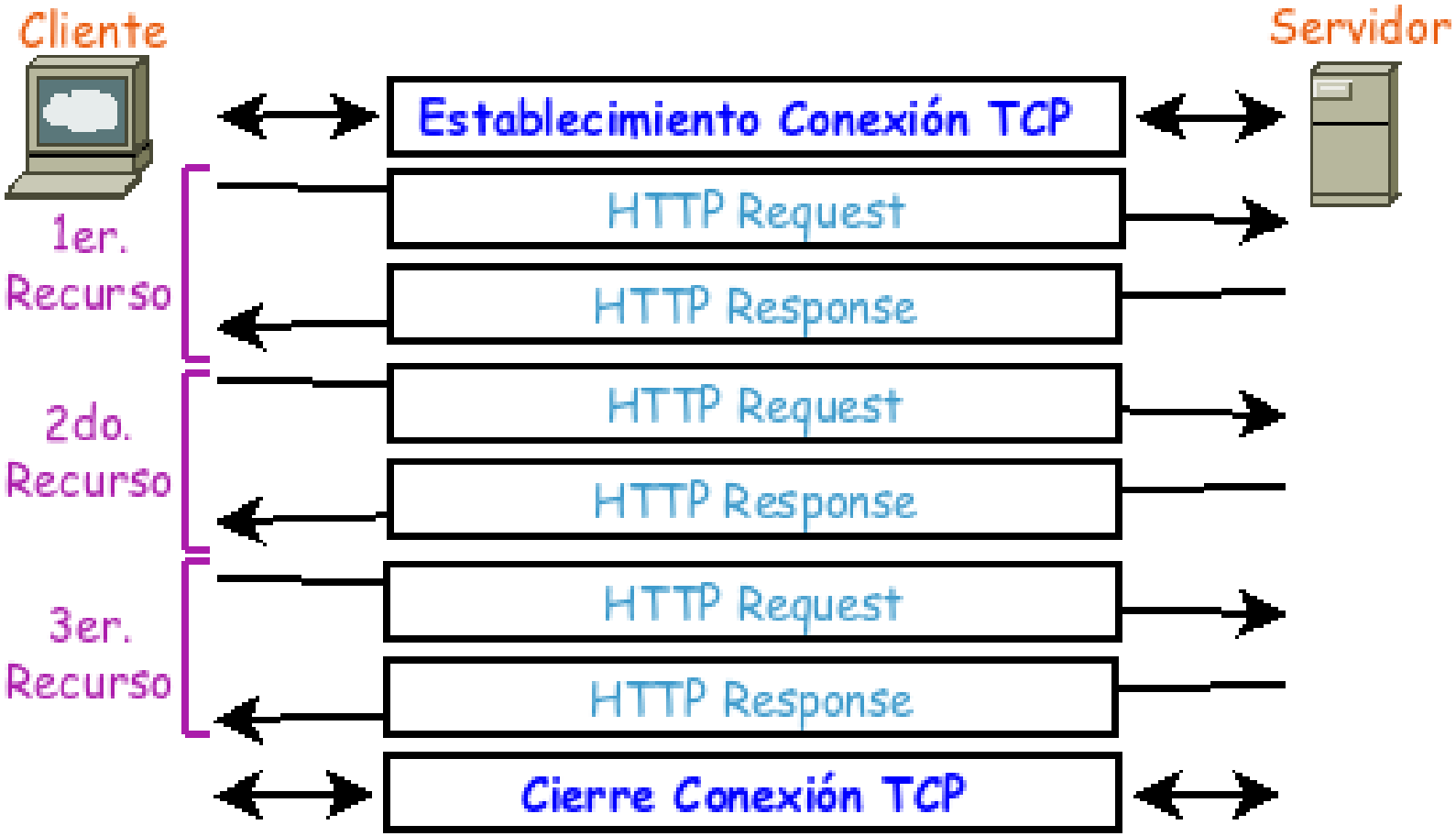
HTTP - Conexiones no persistente



Conexiones persistentes, HTTP/1.1

- Conexiones son persistentes por defecto.
- En una misma conexión TCP se pueden enviar y recibir varios recursos.
- Con la cabecera *Connection: close* se indica que la conexión se debe cerra después de la respuesta del servidor.
- Mensajes debe contener una forma de determinar la longitud exacta de cada mensaje.
- Un extremo puede cerrar la conexión en cualquier momento.

HTTP Persistente



Pipelining - Concurrencia

- **Pipelining**
  - Cliente puede enviar varias solicitudes sin esperar por cada respuesta.
  - Servidor puede procesar esas solicitudes en paralelo si son métodos seguros.
  - Debe enviar cada una de las respuestas en el mismo orden que recibió las solicitudes.
  - No ha sido implementado en forma masiva.
- **Concurrencia**
  - Clientes pueden abrir conexiones simultáneas a un servidor.
  - Son usadas principalmente para evitar el problema del *head-of-line blocking*.
  - RFC no indica la cantidad máxima de conexiones concurrentes, pero pide que las implementaciones sean conservativas.

GET Condicional

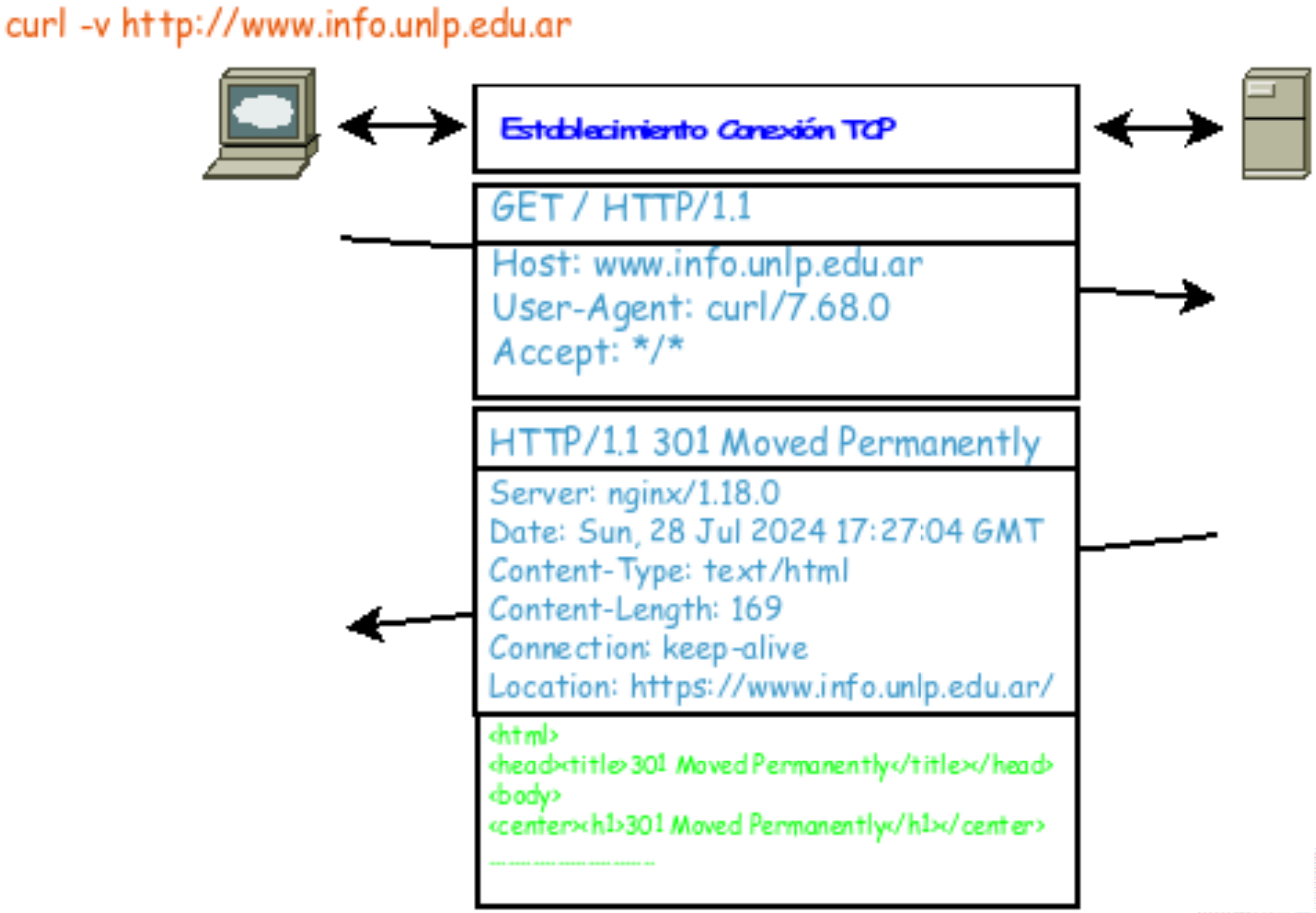
- Uso de la cabecera *If_Modified_Since*.
- El servidor retorna el recurso si fue modificado en una fecha posterior a la indicada en la cabecera.
- Si no se modificó, el servidor responde con un código 304 y el recurso se recupera de la cache.

```
GET /index.html HTTP/1.1
Host: www.unlp.edu.ar
If-Modified-Since: Fri, 22 May 2009 13:53:40 GMT

HTTP/1.0 304 Not Modified
Date: Thu, 21 May 2009 13:55:13 GM
Content-Type: text/html
Expires: Fri, 26 Jun 2009 13:55:13 GMT
```

Redirects

- Si al recurso se le ha asignado una nueva URL, el servidor responde con el código 301(Moved Permanently) o 302 (Found).
- Servidor agrega una cabecera *Location* con al nueva URL.



Estamos en:

1 Introducción

2 Mensajes y Métodos

3 Funcionamiento

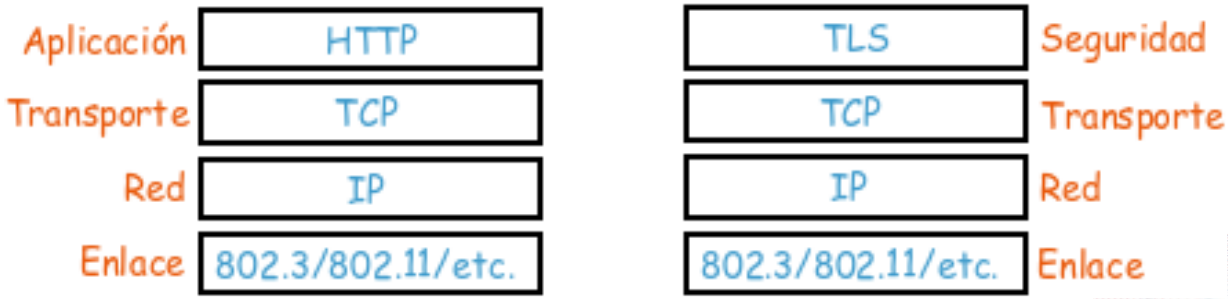
4 HTTP Seguro

5 Cookies

6 HTTP Caching

HTTPS - HTTP sobre TLS

- HTTPS es enviar mensajes HTTP sobre una conexión asegurada por medio de TLS(Transport Layer Security).
- Cliente inicia la conexión al puerto 443, por defecto.
- Se incluye una etapa de negociación previa antes de que HTTP envíe su primera solicitud.
- Todos los mensajes HTTP completos se encriptan y autentican. Viajan como datos de aplicación de TLS.
- Esquema (scheme) indicado en la URL es https://. Por ej.: https://www.info.unlp.edu.ar.



HTTPS - HTTP sobre TLS

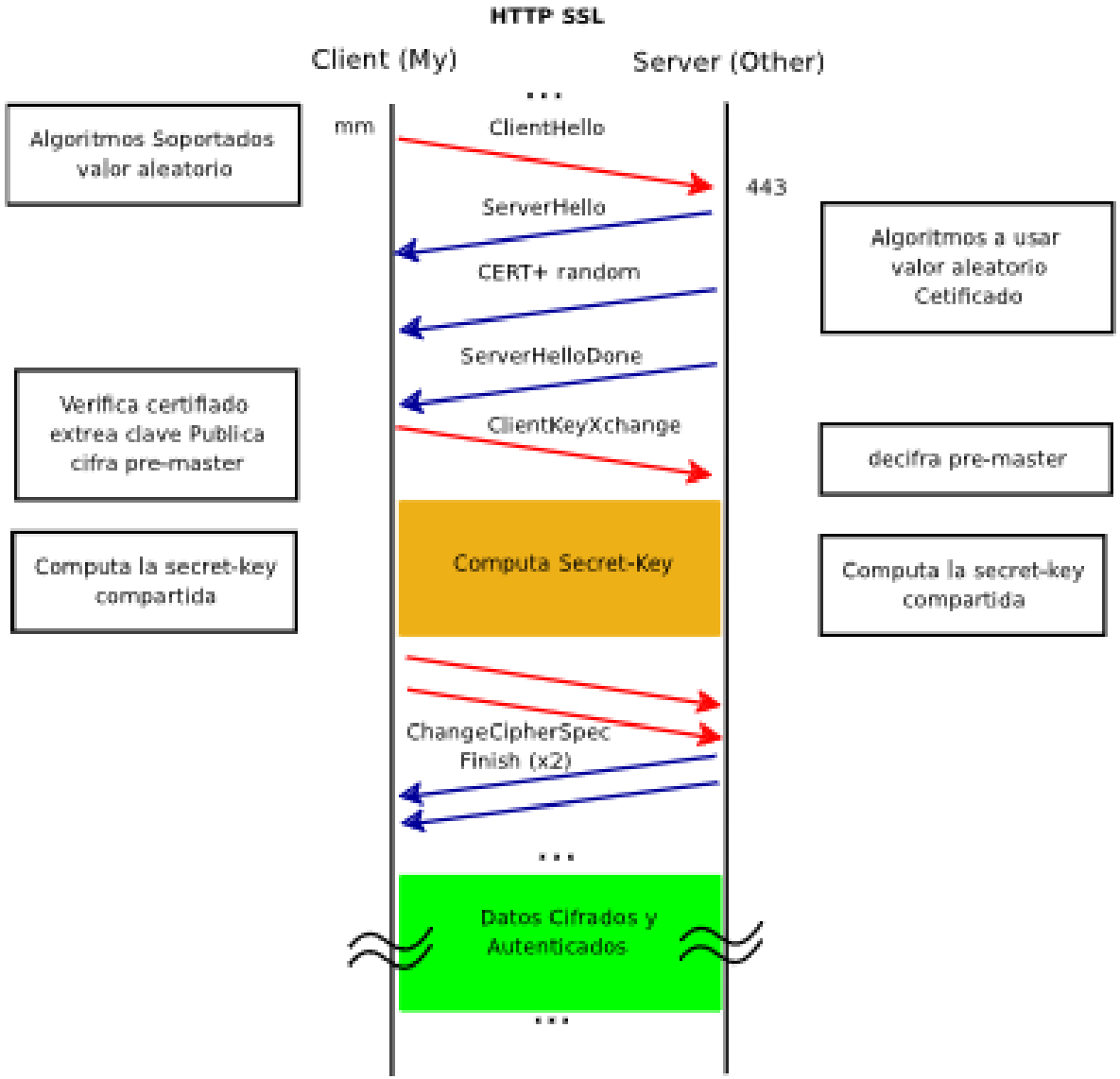


Imagen: Gentileza Andrés Babieri

Estamos en:

1. Introducción
2. Mensajes y Métodos
3. Funcionamiento
4. HTTP Seguro
5. Cookies
6. HTTP Caching

Cookies

- Mecanismo que permite a las aplicaciones web del servidor “manejar estados”.
- HTTP es un protocolo sin estados (no almacena el estado de la sesión entre peticiones sucesivas).
- Las cookies pueden usarse para asociar estados. Proporcionan una manera de conservar cierta información entre peticiones del cliente.
- Servidores envían información de estado al cliente mediante una cabecera *Set-Cookie*.
- En las siguientes solicitudes, el cliente retorna una cabecera *Cookie* con los datos recibidos en la cabecera *Set-Cookie* previa.
- Las cookies contienen una arbitraria lista de información del tipo ”name=value”. .
- Browsers recuerda el contenido de las cookies enviada por un servidor y las puede reutilizar cuando se acceda nuevamente a ese servidor.

Cookies - Usos

- Almacenar información importante para el servidor
- Constituyen una potente herramienta empleada por los servidores Web para almacenar y recuperar información acerca de sus visitantes
- Ejemplos de uso:
  - Guarda información de la sesión
  - Comercio electrónico
  - Carrito de la compra
  - Personalización de páginas
  - Idiomas
  - Seguimiento de las visitas a un Web
  - Carteles publicitarios
  - Almacenamiento del login y password

Cookie enviada por el servidor

- Cabecera HTTP: *Set-Cookie*
  - Cabecera incluida por el servidor en el mensaje de respuesta, cuando quiere enviar una cookie
  - Formato:

    Set-Cookie:
    nombre=valor: Nombre de la cookie y valor
    ;*expires=fecha*: Fecha de caducidad
    ;*path=camino*: Camino
    ;*domain=dominio*: Dominio
    ;*secure*: sólo se transmite sobre canales seguros(HTTPS)

✓ Ejemplo:
Set-Cookie: unnombre=unvalor; expires=Mon, 30-Jan-2001 12:35:23 GMT; path=/dir; domain=mi.dominio.com; secure

Cookie enviada por el cliente

- Cabecera HTTP: *Cookie*
  Enviará todas las cookies en una única cabecera HTTP:
  Cookie: nombre1=valor1; nombre2=valor2;...

- Proceso:
  - Cuando un cliente solicita una URL, buscará en su lista de cookies aquellas que coincidan con ese dominio y con ese camino
  - Dentro de la cabecera *Cookie*, las cookies se ordenan de más a menos específica (según camino)
  - No se consideran las cookies caducadas (de hecho, se eliminan)

Cookies – Ejemplo



Estamos en:

1 Introducción

2 Mensajes y Métodos

3 Funcionamiento

4 HTTP Seguro

5 Cookies

6 HTTP Caching

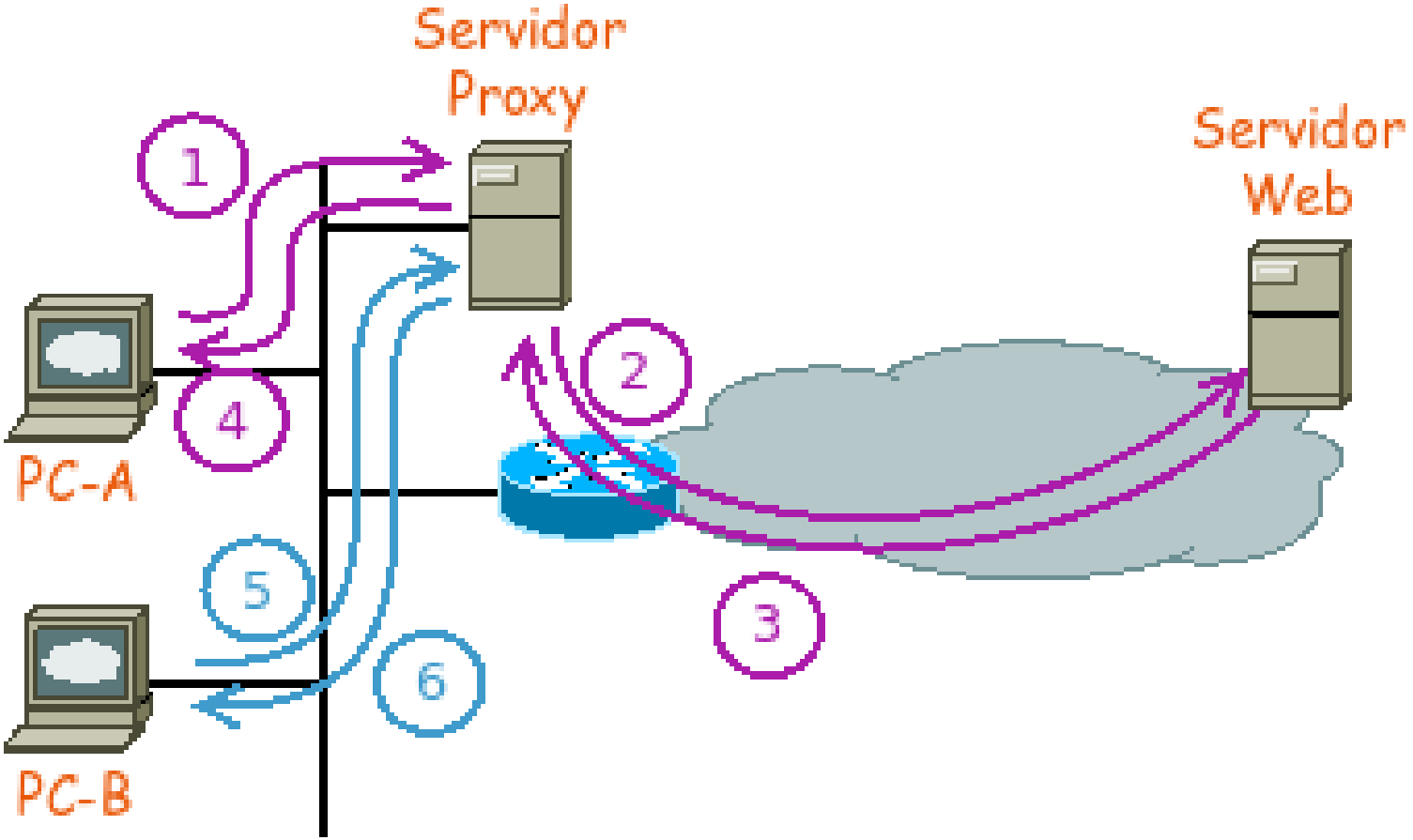
Uso de cachés

- Caching es una característica opcional de HTTP.
- Una cache HTTP consiste de un almacenamiento local de mensajes de respuesta y el sistema encargado de almacenar, recuperar y eliminar los mensajes en ella.
- Almacenan respuestas, susceptibles de ser guardadas, con la intención de reducir el tiempo de respuesta y la carga de la red.
- Necesitan asegurar que los contenidos guardados en la caché sean equivalentes a los almacenados en el servidor.
- Pueden ser privadas, implementadas en el user-agent, o compartidas, implementadas en un servidor proxy.
- En general, almacenan respuestas exitosas de una solicitud de recuperación, pero podrían almacenar redirecciones, resultados negativos (404(Not Found)), etc.

Uso de cachés

- *Cache key* es la información que usa la cache para elegir la respuesta. Mínimamente compuesta por el método y la URL.
- Si la solicitud es servida desde la cache se lo conoce como **Cache hit**.
- Si no se encuentra en la cache, *cache miss* y se debe hacer el pedido al servidor.
- Servidor agrega cabecera, *Last-Modified* (fecha), *ETag*, (entity tag, hash).
- Si el recurso solicitado está en la cache, pero no es válido, se lo debe revalidar
- Para realizar la revalidación utiliza el GET condicional.
- Deben incluir todas las cabeceras recibidas cuando almacena una respuesta.

Funcionamiento de cachés



RFCs – HTTP (166 2022 - 10)

Number	Title	Authors	Date	Status
RFC 1945	**Hypertext Transfer Protocol -- HTTP/1.0**	T. Berners-Lee, R. Fielding, H. Frystyk	May 1996	Informational
RFC 2616	**Hypertext Transfer Protocol -- HTTP/1.1**	R. Fielding, J. Gettys, J. Mogul, H. Frystyk, L. Masinter, P. Leach, T. Berners-Lee	June 1999	Draft Standard
RFC 2660	**The Secure HyperText Transfer Protocol**	E. Rescorla, A. Schiffman	August 1999	Experimental
RFC 2817	**Upgrading to TLS Within HTTP/1.1**	R. Khare, S. Lawrence	May 2000	Proposed Standard
RFC 7034	**HTTP Header Field X-Frame-Options**	D. Ross, T. Gondrom	October 2013	Informational
RFC 7089	**HTTP Framework for Time-Based Access to Resource States -- Memento**	H. Van de Sompel, M. Nelson, R. Sanderson	December 2013	Informational
RFC 7230	**Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing**	R. Fielding, Ed., J. Reschke, Ed.	June 2014	Proposed Standard

RFCs – HTTP

Number	Title	Authors	Date	Status
RFC 7231	**Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content**	R. Fielding, Ed., J. Reschke, Ed.	June 2014	Proposed Standard
RFC 7232	**Hypertext Transfer Protocol (HTTP/1.1): Conditional Requests**	R. Fielding, Ed., J. Reschke, Ed.	June 2014	Proposed Standard
RFC 7233	**Hypertext Transfer Protocol (HTTP/1.1): Range Requests**	R. Fielding, Ed., Y. Lafon, Ed., J. Reschke, Ed.	June 2014	Proposed Standard
RFC 7234	**Hypertext Transfer Protocol (HTTP/1.1): Caching**	R. Fielding, Ed., M. Nottingham, Ed., J. Reschke, Ed.	June 2014	Proposed Standard
RFC 7235	**Hypertext Transfer Protocol (HTTP/1.1): Authentication**	R. Fielding, Ed., J. Reschke, Ed.	June 2014	Proposed Standard
RFC 7236	**Initial Hypertext Transfer Protocol (HTTP) Authentication Scheme Registrations**	J. Reschke	June 2014	Informational

RFCs – HTTP

Number	Title	Authors	Date	Status
RFC 7237	**Initial Hypertext Transfer Protocol (HTTP) Method Registrations**	J. Reschke	June 2014	Informational
RFC 7540	**Hypertext Transfer Protocol Version 2 (HTTP/2)**	M. Belshe, R. Peon, M. Thomson, Ed.	May 2015	Proposed Standard
RFC 7541	**HPACK: Header Compression for HTTP/2**	R. Peon, H. Ruellan	May 2015	Proposed Standard
RFC 7617	**The 'Basic' HTTP Authentication Scheme**	J. Reschke	September 2015	Proposed Standard
RFC 7694	**Hypertext Transfer Protocol (HTTP) Client-Initiated Content-Encoding**	J. Reschke	November 2015	Proposed Standard

RFCs – HTTP

Number	Files	Title	Authors	Date	Status
RFC 7711	ASCII, PDF	**PKIX over Secure HTTP (POSH)**	M. Miller, P. Saint-Andre	November 2015	Proposed Standard
RFC 7725	ASCII, PDF	**An HTTP Status Code to Report Legal Obstacles**	T. Bray	February 2016	Proposed Standard
RFC 7804	ASCII, PDF	**Salted Challenge Response HTTP Authentication Mechanism**	A. Melnikov	March 2016	Experimental
RFC 7807	ASCII, PDF	**Problem Details for HTTP APIs**	M. Nottingham, E. Wilde	March 2016	Proposed Standard
RFC 7838	ASCII, PDF	**HTTP Alternative Services**	M. Nottingham, P. McManus, J. Reschke	April 2016	Proposed Standard
RFC 7840	ASCII, PDF	**A Routing Request Extension for the HTTP-Enabled Location Delivery (HELD) Protocol**	J. Winterbottom, H. Tschofenig, L. Liess	May 2016	Proposed Standard
RFC 7975	ASCII, PDF	**Request Routing Redirection Interface for Content Delivery Network (CDN) Interconnection**	B. Niven-Jenkins, Ed., R. van Brandenburg, Ed.	October 2016	Proposed Standard



Esta obra está sujeta a la licencia Atribución-NoComercial-CompartirIgual 4.0 Internacional (CC BY-NC-SA 4.0) de Creative Commons.

Para detalle de esta licencia visite https://creativecommons.org/licenses/by-nc-sa/4.0/